

Problem 1:

In class, we saw how Ke can prove that she knows the password to a bike chain lock in zero-knowledge. Now, suppose that Ke brings two bike chain locks. She wants to prove in zero-knowledge that she knows the password for at least one of them, but without revealing the password, or revealing which one she knows the password for. Can you design a physical zero-knowledge proof protocol such that Ke can prove the statement in zero knowledge? Please describe the protocol clearly; you can draw a simple picture if that helps. You don't have to prove the security of the protocol.

Proof.

Ke should get two locks and one loop. She should then configure them into Borromean rings. Borromean rings is a knot of 3 rings s.t. if you undo one of the rings the whole thing will undo itself.

For the experiment, Ke first shows you if the locks and loop are in a Borromean rings configuration. Then she will hide it from you unlock one of the locks that she knows. Then all of the locks and loop should fall apart. She should then relock the lock and scramble the code. She will then bring all the things out and show that they are separated. Then she hides it again and relocks it. ■

Problem 2:

Suppose that, as the protocol designer, we already know exactly which subset of nodes are corrupt. Describe a 1-round protocol that achieves Byzantine Broadcast. Here the set of corrupt nodes is provided as input to the protocol. This exercise shows why the problem of Byzantine Broadcast is trivialized if the set of corrupt nodes is known a-priori.

Proof.

Let s be the sender and let B be the subset of corrupt nodes.

First s sends its output to every node and outputs its input.

For all others,

Case 1: $s \in B$ then immediately output 0.

Case 2: $s \notin B$ then output the message received from s .

Consistency:

Case 1: If $s \in B$ then all honest nodes will output the same value 0.

Case 2: If $s \notin B$ then we know s is honest, so it will send the same value to all nodes and all honest nodes will output the given input.

Validity:

If the sender s is honest then we know $s \notin B$, so we will do case 2 in which honest players will output s 's input. ■

Problem 3:

Consider a multi-valued variant of the original Byzantine Broadcast protocol (called Multi-Valued Byzantine Broadcast) in which the sender has an ℓ -bit value $m \in \{0, 1\}^\ell$, and it wants to propagate this value to everyone else. Consistency requires that all honest output the same value; and validity requires that if the sender is honest, all honest nodes output the sender's input value. Design a protocol for achieving Multi-Valued Byzantine Broadcast, and prove that it is secure. Your protocol should be able to tolerate $f < n$ number of corruptions.

Proof.

We can use exactly the Dolev-Strong 1-bit broadcast protocol from class, but treat the sender's value as an ℓ -bit string instead of a single bit.

Let f be an upper bound on the number of dishonest nodes.

R0 : Sender signs and sends a message $\langle m \rangle_1$ to everyone. Add m to extr_1 .

For each round $r = 1$ to $f + 1$:

For every message $\langle \tilde{m} \rangle_{1,j_1,\dots,j_{r-1}}$ player i receives with r signatures from distinct other nodes, including the sender:

- If $\tilde{m} \notin \text{extr}_i$, add $\tilde{m}$ to extr_i . Send $\langle \tilde{m} \rangle_{1,j_1,\dots,j_{r-1},i}$ to everyone.

At the end of round $f + 1$: If $|\text{extr}_i| = 1$, output the message in extr_i . Otherwise, output 0^ℓ .

Claim 1: For $r \leq f$, if m is in some honest player's extracted set by the end of round r , then m will be in every honest node's extracted set by the end of round $r + 1$.

Assume $m \in \text{extr}_i$ for honest player i by the end of round $r \leq f$. This implies there exists an earlier round $r^* \leq r$ where player i added m to extr_i . This means they received $\langle m \rangle_{1,j_1,\dots,j_{r^*-1}}$. This means that player i must have not voted for m before. Thus, $\langle m \rangle_{1,j_1,\dots,j_{r^*-1},i}$ will then be sent to everyone. Thus, every honest node will then also sign this and thus have this message with $r^* + 1$ distinct signatures at the end of round $r^* + 1$.

Claim 2: If some honest node has message m in its extracted set by the end of round $f + 1$, then every honest node has m in its extracted set by the end of round $f + 1$.

Assume $m \in \text{extr}_i$ by the end of round $f + 1$ for honest player i .

Case 1: If player i has $m \in \text{extr}_i$ by $r \leq f$, then by claim 1 every honest node has b in their extracted set by $r + 1 \leq f + 1$.

Case 2: If player i only added $m \in \text{extr}_i$ in round $f + 1$. This implies player i received $\langle m \rangle_{1,j_1,\dots,j_f}$, but since the number of corrupted players is $\leq f$ this implies some honest

player $j \in \{1, j_1, \dots, j_f\}$ signed m before. Thus, j must have signed it earlier, so by claim 1 again every honest node would have m in its extracted set.

Consistency:

By claim 2, all honest players will have the same extracted set at the end of round $f + 1$.

Validity: If the sender is honest with input m , then every honest node outputs m .

Assume sender 1 is honest and has input m .

By the protocol, in round 0 the sender sends $\langle m \rangle_1$ and never signs any other value, since it would not include the sender's signature, so any valid signature chain that includes the sender's signature must be on the same value m . Again by the protocol, a message $\langle \tilde{m} \rangle_S$ where S is a list of signatures is only processed only if $1 \in S$. So a node can add $\tilde{m}$ to extr_i only if there is a valid chain including sender 1's signature. Since signatures are unforgeable by assumption and sender 1 only signed m , this implies that for all i , $\text{extr}_i \subset \{m\}$.

By claim 1, since in round 1 sender 1 adds m to its extracted set this implies that every honest node added m to its extracted set by the end of round 2.

Therefore, putting these together we get that all honest nodes have m in their extracted sets by the end of round $f + 1$, and $|\text{extr}_i| = 1$ for all i . Thus, all honest nodes will output m . ■

Problem 4:

Consider an execution of the Streamlet protocol where $n = 99$. Suppose strictly fewer than $n/3$ nodes are corrupt. Let

$$\perp - e_1 - e_2 - \dots$$

denote a notarized chain starting with the genesis block $\perp$ followed by blocks with epoch numbers e_1, e_2 , and so on. For each of the following scenarios, please answer whether such a scenario can possibly occur during the execution. If so, please provide an explanation of how such a scenario can occur, and state which blocks are considered final by honest nodes. If not, please explain why.

(a)

The two *notarized* chains $\perp - 1 - 3 - 5 - 7$ and $\perp - 1 - 2 - 4 - 6$ will both be in honest view.

Proof.

Yes this can happen due to network delay.

Let 32 nodes be corrupt. Let A be 33 honest nodes. Let B be 33 honest nodes disjoint from A , and let c be the final remaining honest node.

In epoch 1 all nodes vote to notarize 1, and they all get back the confirmation.

In epoch 2 all corrupt nodes, all nodes in A , and c vote to extend the chain and get 1 – 2. This is 66 votes. All but c get back the confirmation it is notarized in epoch 2. B sees nothing.

In epoch 3 all corrupt nodes, all nodes in B and c vote to extend the chain 1 – 3. This is 66 votes. Everyone gets back the confirmation it is notarized and then c also gets back that 1 – 2 gets notarized. A sees nothing.

You can then repeat this.

Also, no blocks are finalized. ■

(b)

An honest node P observes a notarized chain of the form $\perp - 1 - 2 - 3 - 4$ and an honest node Q observes a notarized chain of the form $\perp - 1 - 2 - 3 - 5$.

Proof.

Yes this can happen due to network delay.

Let 32 nodes be corrupt. Let A be 33 honest nodes. Let B be 33 honest nodes disjoint from A , and let c be the final honest node.

First let all nodes see the notarized chain $1 - 2 - 3$.

In epoch 4, let all corrupt nodes, A , and c vote to notarize $1 - 2 - 3 - 4$. All nodes except c get confirmation. B sees nothing.

In epoch 5, let all corrupt nodes, B , and c vote to notarize $1 - 2 - 3 - 5$. All nodes except c get confirmation. A sees nothing.

So for all nodes in A see $1 - 2 - 3 - 4$ notarized, and all nodes in B see $1 - 2 - 3 - 5$ notarized.

1-2-3 are finalized. ■

(c)

An honest node P observes a notarized chain of the form $\perp - 1 - 2 - 3 - 4 - 5$ and an honest node Q observes a notarized chain of the form $\perp - 3 - 4 - 5$.

Proof.

This is not possible.

This implies that $\perp - 1 - 2 - 3$ and $\perp - 3$ was notarized at epoch 3 which is a contradiction since only one block can be notarized per epoch. ■

Problem 5:**(a)**

Alice is asked to implement the Dolev-Strong protocol for 475 course. Alice forgot to check that the batch of r signatures expected in round r must contain the signature from the designated sender. Describe an attack that can break Alice's implementation. In your attack, use as few corrupt nodes as possible. Does your attack violate the consistency or validity property?

Proof.

Use one corrupted node.

Let the sender be honest and get an input bit 1.

In round 0: Honest sender sends $\langle 1 \rangle_1$ to everyone.

Then make the corrupted node send $\langle 0 \rangle_c$ s.t. c is its own signature to everyone.

Thus, on round 1 every honest node i now has two messages in their extracted set's which breaks validity since all the honest node's will not output 1 even though its sender was honest. ■

(b)

Another student, Bob, also made a mistake. Bob has a bug in implementing the digital signature scheme. As a result, an attacker can *efficiently* forge signatures of honest players even without their private signing keys. Please describe an attack that breaks Bob's Byzantine Broadcast implementation.

Proof.

We can again use one corrupted node.

Let the sender be honest and get an input bit 1.

In round 0 the honest sender first sends $\langle 1 \rangle_1$ to everyone.

Then make the corrupted node send $\langle 0 \rangle_1$ to everyone which it can do since the corrupted node can forge the senders signatures.

Thus, on round 1 every honest node will now have two messages in their extracted set's which breaks validity since all the honest node's will not output 1 even though the sender was honest. ■